Overview: Local (air-gapped) Nextcloud with Euro-Office under Podman

This document gives an overview of how Nextcloud and Euro-Office were installed on this host as a **fully local** (air-gapped) instance that is reached through the local domain `nextcloud.local`. It is the entry point for the detailed records in this directory:

Document	Content
installation-steps.md	Full install log of Nextcloud All-in-One with Podman
local-nextcloud-fixes.md	The Podman / local-only fixes in detail, plus automation
podman-vs-docker.md	Why Podman is preferred for a government agency
nextcloud-euro-office-integration.md	Euro-Office integration and the document-saving fix
combining-nextcloud-euro-office.md	How Nextcloud AIO bundles Euro-Office as an option
euro-office-github.md	Euro-Office install guide (upstream, GitHub-based)

1. Goal and why a local domain

The aim was a Nextcloud instance that runs **entirely on the local network, with no internet link** - an air-gapped installation. A public domain name can therefore never be used: no Let's Encrypt certificate can be issued for a LAN-only name, and no external service may be contacted during setup.

To still give the instance a stable, human-friendly address, the local domain **`nextcloud.local`** was mapped to the host's LAN IP in `/etc/hosts`:

```
192.168.0.114  nextcloud.local
```

Nextcloud All-in-One (AIO) normally insists on a fully qualified public domain and validates it against the public DNS. For a local-only install this is disabled with `SKIP_DOMAIN_VALIDATION=true`, which makes the AIO interface accept any domain without an internet/DNS check. The same principle is described in domain-validation-nextcloud.md.

2. Approach: GitHub repositories as starting point

Instead of running a ready-made installer script, the **source repositories on GitHub** were taken as the starting point and the upstream documentation was followed:

- nextcloud/all-in-one - the AIO mastercontainer that manages the whole Nextcloud stack
- Euro-Office/DocumentServer - the office document server
- Euro-Office/eurooffice-nextcloud - the Nextcloud integration app

Because Podman is not an officially supported engine for AIO, the community guide Rootless Podman Quadlet was used. The `docker run` command from the AIO README was adapted for Podman; the full command is recorded in installation-steps.md.

3. Installation summary

Environment: Linux Mint 22.3, user `naj` (uid/gid 1000), rootless Podman 4.9.3, host LAN IP `192.168.0.114`.

1. **Prerequisites**: rootless Podman with cgroups v2, the Podman API socket (`podman.socket`, exposes the Docker-compatible API that AIO requires), the `podman-restart.service` user unit, and the pulled mastercontainer image.
2. **Mastercontainer**: created with the AIO `docker run` command adapted for Podman (podman socket mount, `DOCKER_API_VERSION=1.41`, `--network bridge`, `SKIP_DOMAIN_VALIDATION=true`).
3. **Browser setup**: the AIO interface is reached at `https://192.168.0.114:8080` (by IP, never by domain, to avoid HSTS issues). The domain `nextcloud.local` is entered without validation, then the required containers (Nextcloud, Database, Redis, Apache) and optional add-ons are started.

4. Fixes required for a functional Nextcloud under Podman

Several things had to be fixed to get the installation to work. They are documented in detail in local-nextcloud-fixes.md and collected in the troubleshooting section of installation-steps.md:

1. **Podman socket permissions** - AIO's `www-data` user must be able to talk to the container engine socket. The socket had been created with mode `0600` instead of the declared `0660`; `systemctl --user restart podman.socket` recreates it correctly.

2. **Docker API version mismatch** - AIO defaults to Docker API `v1.44`, while Podman 4.9.3 exposes `v1.41`. Without `DOCKER_API_VERSION=1.41` the mastercontainer refuses to start.

3. **Non-root containers cannot bind port 443** - Docker grants non-root processes `CAP_NET_BIND_SERVICE`; Podman grants no effective capabilities to non-root users by default, so the `www-data` user in the domaincheck/apache containers got `bind() [::]:443: Permission denied`. Fix: patch `containers.json` inside the mastercontainer

to add `CAP_NET_BIND_SERVICE` to those two containers. (Setting `net.ipv4.ip_unprivileged_port_start=80` via `default_sysctls` does not help - the Docker-API path ignores it.)

4. **Self-signed TLS for a local-only domain** - the apache container's Caddy tries to obtain a Let's Encrypt certificate for `nextcloud.local`, which can never succeed, so HTTPS fails with `SSL_ERROR_INTERNAL_ERROR_ALERT`. Fix: bind-mount a custom Caddyfile template that uses Caddy's **internal CA** (`tls { issuer internal }`), which issues a self-signed certificate valid for the domain.
5. **Stale php-fpm `listen.allowed_clients`** - recreating the apache container gives it a new IP in the `nextcloud-aio` bridge network; the Nextcloud container only accepts php-fpm connections from the apache IP recorded at its own start, causing HTTP 503. Fix: `podman restart nextcloud-aio-nextcloud`.

The first four items are applied automatically by the helper script `~/.local/bin/nextcloud-aio-podman-fix.sh`, which is installed as the systemd user service `nextcloud-aio-podman-fix.service` and runs at every session start. It patches `containers.json` idempotently and restarts the mastercontainer only when something changed.

> [!WARNING] The `containers.json` patches are **lost whenever the mastercontainer is recreated** (e.g. when updating). Re-run the helper script afterwards.

5. Why Podman is the better choice for a government agency

Podman was chosen over Docker deliberately; podman-vs-docker.md argues the case in full. The short version:

- **Least privilege / compliance** - Podman is **rootless by default** (user namespaces), so containers run as unprivileged users - no root daemon socket to grant root-equivalent access, satisfying FedRAMP, NIST SP 800-53, DISA STIGs and SOC 2 audits.
- **Smaller attack surface** - Podman is **daemonless** (fork/exec model); there is no central `dockerd` whose compromise or crash would affect every container on the host.
- **Zero licensing risk** - Podman is free and open source (Apache 2.0). Docker Desktop requires paid subscriptions for larger organizations, creating audit and procurement overhead.
- **Government Linux alignment** - Podman is the **default container engine in RHEL 8+**, is FIPS-compliant, and integrates natively with `systemd` (this installation uses `systemd` user units and user linger throughout).

- **Auditability** - because containers run as child processes of the invoking user, `auditd` can track individual user IDs down to container events, simplifying forensics.
- **Drop-in compatibility** - Podman is OCI-compliant, so the same images and `Containerfile`/`Dockerfile` workflows keep working (`alias docker=podman`).

6. Euro-Office integration

The last step was integrating **Euro-Office**, an office suite tailored for government agencies. It has stripped out all suspect software and linkages found in other office suites, making it suitable for an air-gapped government environment. See nextcloud-euro-office-integration.md for the full record.

Euro-Office is deployed in two parts (see euro-office-github.md):

1. **Document server** - the real-time collaboration server for office documents.
2. **Nextcloud app** (`eurooffice`) - the integration layer that opens and saves documents inside Nextcloud.

Because AIO ships Euro-Office as a selectable add-on container (combining-nextcloud-euro-office.md), the document server runs as the AIO-managed container `nextcloud-aio-eurooffice` on the `nextcloud-aio` network, reachable in the browser at `https://nextcloud.local/eurooffice`. The app is configured with `DocumentServerUrl = https://nextcloud.local/eurooffice` and a shared JWT secret injected by AIO.

The saving fix

Opening documents worked, but **saving failed** with "The document cannot be saved". The documentserver's background services (converter, docservice) call back into Nextcloud over `https://nextcloud.local`, which is served with a **self-signed certificate** and resolves to a **private IP** - the TLS handshake failed with `UNABLE_TO_GET_ISSUER_CERT_LOCALLY`.

Fix: two settings that map to the documentserver's `local.json`:

Env var	Effect
`USE_UNAUTHORIZED_STORAGE=true`	`requestDefaults.rejectUnauthorized = false` (accept the self-signed certificate)
`ALLOW_PRIVATE_IP_ADDRESS=true`	`request-filtering-agent.allowPrivateIPAddress = true` (allow the private IP as callback target)

These were applied durably in `containers.json` (survives container recreates) and immediately in the running container's `local.json`. The helper script

`nextcloud-aio-podman-fix.sh` was extended to (re-)apply the two env vars after every mastercontainer recreate.

> [!NOTE] Disabling `rejectUnauthorized` is acceptable only for this local-only setup with a self-signed certificate. If the instance ever moves to a public domain with a real Let's Encrypt certificate, the variables can be removed again so TLS verification stays enabled.

Result

With the fixes applied, Euro-Office documents can be opened and saved from Nextcloud through the local domain `nextcloud.local` - a fully local, air-gapped office and file stack for a government agency, running rootless under Podman.